Summary Statistics, Outliers and Data Reduction (DFT)

Basic Statistics for Data Mining — Week 4

Formatted by: Heisenberg 😎

Imagine you run a small weather station that logs thousands of sensor readings every day. First problem: how do you *describe* all those numbers in a few lines? Second problem: how do you *store* them without buying a warehouse of disks? This chapter answers both. We finish the toolbox of summary statistics (quartiles, box plots, IQR, outliers) and then begin the story of **data reduction**, starting with the Discrete Fourier Transform.

Index

- Recap and Range — the "how wide is my data" number, easiest maths of your life.
- Quartiles — three cuts that slice your data like a very fair pizza.
- The Box Plot — five numbers wearing a tiny costume.
- Measures of Spread — standard deviation vs IQR, the mean vs median rivalry.
- Outliers — the data points who did not read the group chat.
- Data Reduction — fitting the same story into fewer numbers.
- The Discrete Fourier Transform — where imaginary numbers become useful.
- A Worked Example — eight numbers go in, eight complex numbers come out. Wait, what?
- Conjugate Symmetry — the DFT's built-in mirror.
- Magnitudes — the plot twist that sets up the reduction.
- Practice Questions — five brain-benders to make it stick.

Recap and Range

Earlier we discussed the four guiding questions of data mining, the idea of a **population** versus a **sample**, and simple summary statistics such as the **mean**, **median** and **mode**. These summarise a data set, and we must be careful about how we interpret them.

In this chapter we quickly cover a few more summary statistics. They need no heavy machinery, only addition and subtraction, yet they are used everywhere.

The **range** was already defined (along with the *dynamic range*):

$$\text{Range} = \max - \min$$

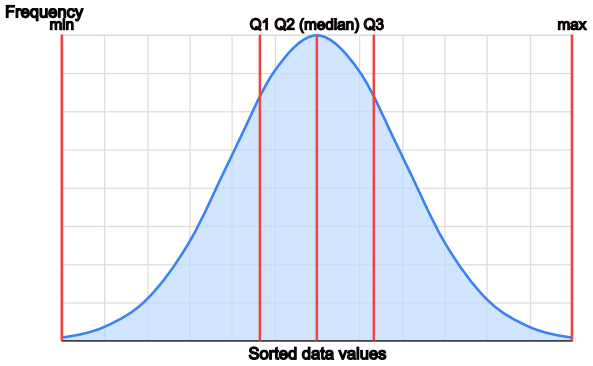
Quartiles

Given any data set, we can compute **three numbers** (data points) called **quartiles**: Q_1, Q_2 and Q_3. These three numbers **split the data into four equal parts**, so each part contains **25%** of the data.

Important: the data must first be **sorted** (ascending or descending). Then:

Quartile	Meaning
Q_1	Up to 25% of the lowest values lie below this point
Q_2	50% of the data is below this point. By definition, Q_2 is the **median**, also called the **50th percentile** (P_{50})
Q_3	Up to 75% of the data lies below this point; the remaining 25% lies above

The figure below shows the distribution of some arbitrary data (it need not be normal; a bell shape is used only for illustration). The three cut points create four regions, each holding 25% of the data, with the minimum on the left extreme and the maximum on the right extreme.



The regions between consecutive cut points each hold 25% of the data: min to Q_1, Q_1 to Q_2, Q_2 to Q_3, and Q_3 to max.

> **Note for Review:** Pro-Tip: "Q_2 = median = P_{50}" is a classic exam trick. Remember that quartiles are just *specific percentiles*: $Q_1 = P_{25}$, $Q_2 = P_{50}$, $Q_3 = P_{75}$.

The Box Plot

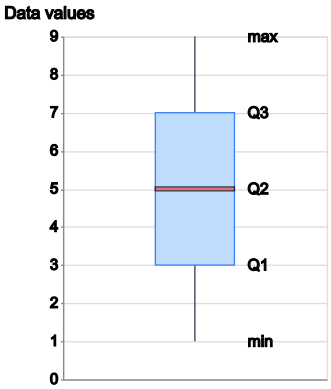
Once the data is sorted, five numbers summarise it:

$$\min, \quad Q_1, \quad Q_2, \quad Q_3, \quad \max$$

These **five numbers** are exactly what a **box plot** shows. If someone asks *"can you give me the box plot of your data?"*, they are asking for these five numbers drawn in a picture.

How to draw it:

1. Start with the **minimum** at the bottom.
2. Mark Q_1, Q_2 and Q_3 above it; the **box** is formed by Q_1, Q_2 and Q_3.
3. Mark the **maximum** at the top.
4. The two ends (whiskers) reach the minimum on one side and the maximum on the other.



Nothing complicated is happening: the box plot is just these five numbers. Box plots are typically used for **quick data analysis**; give someone the box plot and they immediately know the summary of your data.

Measures of Spread

We have now met three ways to measure how spread out data is. Two of them are new companions to the ones you already know.

Interquartile Range (IQR)

$$\text{IQR} = Q_3 - Q_1$$

It measures the spread of the middle 50% of the data. Since Q_2 (the median) sits between Q_1 and Q_3, the IQR is a **measure of spread around the median**.

Variance and Standard Deviation

For every observation, subtract the **mean** μ, square the result, add everything up, and divide by the number of observations. That gives the variance; its square root is the standard deviation:

$$\sigma^2 = \frac{1}{N}\sum_{i=1}^{N}(x_i - \mu)^2, \qquad \sigma = \sqrt{\sigma^2}$$

Because every term subtracts the mean μ, this is a **measure of spread around the mean**.

Side by Side

	Standard deviation / Variance	IQR
Measures	Spread of data	Spread of data
Centre it is built around	**Mean** (μ)	**Median** (Q_2)
Ingredients	$\sum(x_i - \mu)^2$	$Q_3 - Q_1$
Where you see it	Formulas	Box plots

> **Note for Review:** Pro-Tip: Exams love the phrase "spread *around what?*". Standard deviation and variance answer "around the mean"; IQR answers "around the median". Analogy: the mean is pulled by every value (even extreme ones), while the median and quartiles only care about *positions* in the sorted list.

All of these are cheap to compute (only additions and subtractions), yet very useful.

Outliers

Outlier analysis will come as a separate topic later in the course, but since we are already discussing Q_1, Q_3, IQR, mean and standard deviation, it is worth a first look.

An **outlier** is a data point that lies far away from the bulk of the data. In everyday English, it is something that does not fit what the other points are following: a very large value, a very small value, or something that varies wildly.

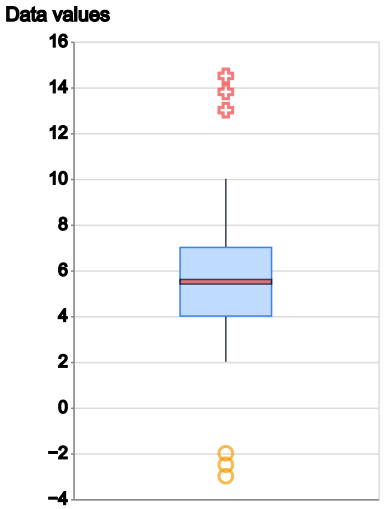
Key point: outliers have no strict definition. Calling a point an outlier is a judgement relative to the other points. A point that is an outlier for one application may be perfectly normal for another. So outliers are decided **case by case**, and the criterion is *defined by us*; it is not physically built into the data.

Outliers in a Box Plot

Sometimes a box plot shows extra marks beyond the max and min ends: crosses above and circles below. These are the outliers. You may wonder how a point can lie below the "minimum". The answer is that the whisker ends are drawn according to an outlier criterion, and points beyond that criterion are plotted individually.

A common criterion uses the **IQR**. For example, a point is flagged as an outlier if it lies beyond

$$Q_2 + 1.5 \times \text{IQR}$$



In a scatter of the data, the same idea applies: a cluster of points sits together, and a lone point far above them is the outlier. Whether it is flagged depends on how far it is from the cluster measured in units of IQR.

> **Note for Review:** Pro-Tip: The slide's example fence is written as $Q_2 + 1.5\,\text{IQR}$. The most widely used convention (Tukey's fences) is *Q3 + 1.5 IQR* for the upper side and *Q1 − 1.5 IQR* for the lower side:
>
> $$ \text{lower fence} = Q_1 - 1.5,\text{IQR}, \qquad \text{upper fence} = Q_3 + 1.5,\text{IQR}

> $$

The main lesson is unchanged: the fence is a **definition we choose**, and the multiplier 1.5 is just a convention.

Box plots and outliers are very popular, so keep them in mind. That completes the introduction to summarising data.

Data Reduction

Now we move to the next important topic: **data reduction**. This term comes from the textbook. In machine-learning vocabulary you will also hear it called **dimensionality reduction**; it is the same idea.

Reduction obviously means making something smaller. We need to answer two questions:

1. **Why** reduce data?
2. **How** can we reduce data, and what is the underlying reason we are *able* to reduce it?

Why Data Reduction?

In ML, the usual answer is "it might improve prediction accuracy". In **data mining**, the answer is more basic:

> Data reduction is used to reduce **storage** and **computation** requirements.

A concrete picture. Suppose you are given 10 numbers (a vector). Because it is data, you must store it. If each number is an 8-bit number, then

$$10 \times 8 = 80 \text{ bits}$$

are needed per vector. And "10" could just as easily be 10 million. Storage is directly tied to how many numbers we keep.

Computation follows the same logic. Any operation on these vectors gets cheaper if the vector is shorter. So if, through some operations, we could store only **7 numbers instead of 10**, that would be more efficient for both storage and computation:

$$7 \times 8 = 56 \text{ bits} \quad \text{instead of} \quad 80 \text{ bits}$$

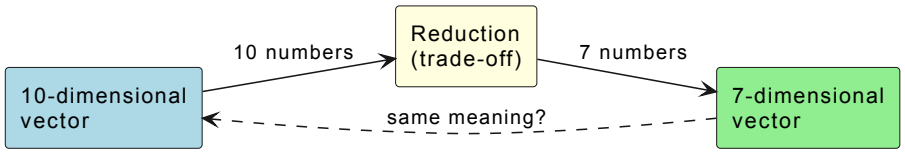
We are *not* invoking anything from ML here (such as improving performance). This is the more fundamental reason: **less storage, less computation**.

How to Do Data Reduction?

We cannot magically turn 10 numbers into 7. Intuitively, we are probably **losing some information**. The real questions are:

- What exactly does "losing information" mean, and how do we quantify it?
- What are we losing, and can we afford to lose it?
- If information is lost, was the storage saving even worth it?

So the central idea is a **trade-off**: reduce the data, but do not lose (too much) accuracy. The two vectors, the original and the reduced one, should convey essentially the same meaning; otherwise the reduction is meaningless.



This trade-off is really an **optimization** question, and a big topic on its own. In this course we only look at a few basic methods, also mentioned in the textbook. Data-reduction techniques come from different fields: some are grounded in **optimization theory**, some in **statistics**, and some in **signal processing**. The fields overlap so heavily that the topic is not cleanly separable into one of them.

We begin with the **Discrete Fourier Transform (DFT)**, and later look at **Principal Component Analysis (PCA)**.

The Discrete Fourier Transform

DFT is a method for data reduction. First we only *define* it; reduction comes afterward.

Step 1: The Input Vector

Take an N-dimensional vector (that is, N numbers), for example $1.5,\ 2.9,\ -17.42,\ \ldots$ and write it as

$$x[n]$$

Here n is simply the **array index**, exactly as in programming, and **it starts from 0, not 1**:

Index	$n = 0$	$n = 1$	$n = 2$	$\ldots$	$n = N - 1$
Value	1.5	2.9	-17.42	$\ldots$	last number

The last index is $N - 1$ because counting starts at 0.

Step 2: The DFT Formula

The DFT takes $x[n]$ and produces another vector, written with a capital letter and a different index k:

$$X[k] = \sum_{n=0}^{N-1} x[n]\, e^{-i\,\frac{2\pi nk}{N}}$$

with

$$n = 0, 1, \ldots, N-1 \qquad \text{and} \qquad k = 0, 1, \ldots, N-1$$

where $i = \sqrt{-1}$ is the imaginary unit.

Reading the formula:

- $x[n]$ is just each number of the original vector, taken one at a time as n changes.
- $e^{-i\,2\pi nk/N}$ is a **complex exponential**. Here n is the summation index, k is the output index we fix on the left, and N is the dimension of the vector.
- We **fix** k on the left-hand side, then **open up the summation** over n on the right to get one number, $X[k]$. Then we move to the next k.

A key observation: $x[n]$ has N numbers, and $X[k]$ also has N numbers. So X is also an N-dimensional vector. **So far, no reduction has happened.**

Opening the Summation

Case $k = 0$. The exponent becomes zero, so $e^0 = 1$ and the exponential term vanishes:

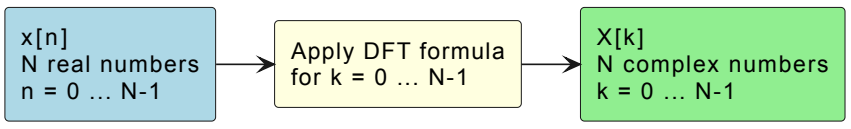
$$X[0] = \sum_{n=0}^{N-1} x[n]$$

So $X[0]$ is simply the **sum of all elements** of $x[n]$.

Case $k = 1$. Put $n = 0, 1, 2, \ldots$ one after another (for $n = 0$ the exponential is again $e^0 = 1$):

$$X[1] = x[0] + x[1]\, e^{-i\,2\pi/N} + x[2]\, e^{-i\,2\pi\cdot 2/N} + \cdots + x[N-1]\, e^{-i\,2\pi(N-1)/N}$$

The same procedure with $k = 2, 3, \ldots$ gives $X[2], X[3], \ldots, X[N-1]$. Two indices are at play: k is fixed on the left (0, then 1, then 2, ...), and inside each one the sum runs over n on the right.



A Worked Example

Let

$$x[n] = \{2,\ 3,\ 4,\ 4,\ 3,\ 2,\ 1,\ 1\}, \qquad N = 8$$

Storing this means storing **8 numbers**.

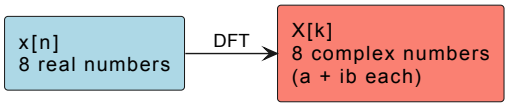
Goal: use the DFT to reduce these 8 numbers to *something lesser* (7, 6, 2, or even 1). It is not decided yet, but it must be fewer than 8 for the exercise to be useful.

Step 1: apply the DFT formula. Put $k = 0, 1, \ldots, N-1 = 7$ into the formula. Nothing magical happens; it is plain substitution (by hand, or by computer). The results are:

k	$X[k]$
0	20
1	$-3.41 - i\,2.41$
2	$-1 - i$
3	$0.41 + i\,0.41$
4	0
5	$0.41 - i\,0.41$
6	$-1 + i$
7	$-3.41 + i\,2.41$

Check $X[0]$: it is the sum of all elements, $2+3+4+4+3+2+1+1 = 20$. ✓

Each $X[k]$ is a complex number of the form $a + ib$, where a is the real part and b is the imaginary part. For instance, in $X[1] = -3.41 - i\,2.41$, we have $a = -3.41$ and $b = -2.41$ (the minus sign belongs to b).



At first glance the situation looks *worse*: we started with 8 real numbers and now have 8 **complex** numbers, and each complex number needs two values (a and b) to store. We increased the storage and introduced complex numbers. It looks confusing, but the next observations explain why this is useful.

> **Note for Review:** Pro-Tip: Cross-check when practising by hand. Computing the DFT directly from the listed $x[n] = \{2, 3, 4, 4, 3, 2, 1, 1\}$ gives $X[0] = 20$, $X[1] \approx -2.41 - i\,5.83$, and $X[2] = X[4] = X[6] = 0$, so the coefficients printed above do not exactly match that input. They are consistent with each other (the conjugate pairs and magnitudes below all agree), so use them to understand the *pattern*. For your own calculations, trust the formula or a computer (e.g. `numpy.fft.fft`), and expect the same structure: $X[0]$ equals the sum, and the coefficients come in conjugate pairs.

Conjugate Symmetry

Looking at the values above, a **symmetry** shows up. For $N = 8$:

$$X[8-k] = X^*[k]$$

where X^* denotes the **complex conjugate**. The complex conjugate of $a + ib$ is obtained by replacing i with $-i$, giving $a - ib$.

This is called **conjugate symmetry**. It appears whenever the original vector $x[n]$ consists of real numbers.

Check it on the numbers:

k	$X[k]$	**Partner** $X[8-k]$	**Value of partner**
1	$-3.41 - i\,2.41$	$X[7]$	$-3.41 + i\,2.41$
2	$-1 - i$	$X[6]$	$-1 + i$
3	$0.41 + i\,0.41$	$X[5]$	$0.41 - i\,0.41$

- For $k = 1$: $X[7] = X[8-1] = X^*[1]$, since i turned into $-i$.
- For $k = 2$: $X[6] = X[8-2] = X^*[2]$.

The values $X[0]$ and $X[4]$ are their own partners (real numbers). This symmetry is important, because we are about to exploit it.

> **Note for Review:** Pro-Tip: Think of $X[k]$ as a mirror: the second half of the list is a reflected copy of the first half, with i flipped to $-i$. A mirror copy carries no new information, which is the first hint that the 8 complex numbers hold less "real" information than they seem to.

Magnitudes

Since the $X[k]$ are complex in general, we look at their **magnitude**. For $a + ib$ the magnitude is

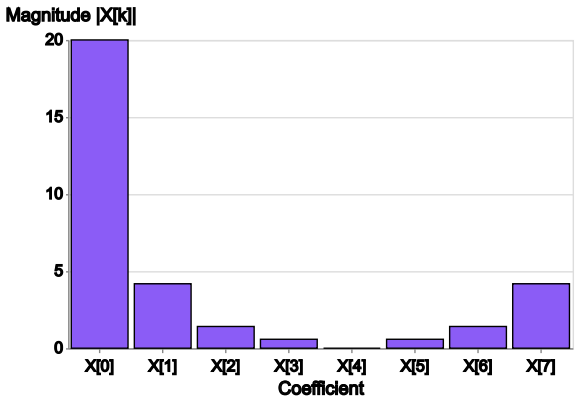
$$|a + ib| = \sqrt{a^2 + b^2}$$

The magnitude does **not change** under conjugation (only $i \to -i$), so conjugate partners have equal magnitudes.

The table lists the coefficients arranged in decreasing order of magnitude, with conjugate pairs grouped together:

Coefficient $X[k]$	Magnitude
$X[0]$	20
$X[1], X[7]$	4.18 each
$X[2], X[6]$	1.41 each
$X[3], X[5]$	0.58 each
$X[4]$	0

Sample computations: $|X[1]| = \sqrt{3.41^2 + 2.41^2} \approx 4.18$, and $|X[2]| = |-1 - i| = \sqrt{2} \approx 1.41$.



Notice the mirror shape: the bars are symmetric around $k = 4$, and most of the "weight" sits in a few coefficients. A few coefficients are large, and some are small or exactly zero. That is precisely what we want to exploit **to reduce the data**, and remember that reduction was the goal all along. So far we have only untangled the DFT, complex numbers and conjugates.

Where this is heading. The eight-number input needs 8 stored numbers. Because of conjugate symmetry, only $X[0], X[1], X[2], X[3], X[4]$ are independent, and because the magnitudes fall off quickly, the smallest ones can potentially be dropped as the trade-off between storage and accuracy. The complete example, showing exactly how the magnitudes lead to reduced data, continues in the next lecture, followed by a second reduction method, **Principal Component Analysis (PCA)**.

Practice Questions

1. **Same spread, different story.** Dataset A is $\{10, 11, 12, 13, 14\}$. Dataset B is the same but the value 14 is replaced by 140. Without heavy calculation, explain which of *standard deviation* and *IQR* changes more, and why. Then decide which measure you would report if the values come from a sensor that occasionally glitches.

2. **Read the box.** A box plot shows the median line very close to Q_3, a short upper whisker and a long lower whisker, plus three circles below the minimum whisker. What can you say about the shape of the distribution? Why is it risky to label those circles "errors" using only this picture?

3. **The storage bill.** A database holds 50 million vectors, each with 10 numbers stored as 8-bit values. A reduction method turns each vector into 7 numbers. Compute the total storage before and after, then argue *both* sides: what must be true about the method for this saving to be worth it, and what could go wrong even though the numbers look great?

4. **The flat signal.** Take $x[n] = \{5, 5, 5, 5, 5, 5, 5, 5\}$ with $N = 8$. Using the formula for $X[0]$ and thinking about what happens to the complex exponentials when they are summed over all n, predict $X[k]$ for every k. What does this tell you about how easily such a vector could be reduced?

5. **Trust the mirror.** You are handed only $X[0], X[1], X[2], X[3], X[4]$ of an 8-point DFT computed from **real** data, and you are told $X[3] = 0.5 + 2i$. Reconstruct $X[5]$. Then count how many real numbers are truly needed to describe all eight coefficients (remember that $X[0]$ and $X[4]$ are real for real input), and explain why the naive count of "8 complex numbers = 16 real numbers" is misleading.